Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп’ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 11

Тема: Об'єктно-орієнтоване проєктування класів та перевантаження магічних методів

Мета роботи: Опанувати практичні навички об'єктно-орієнтованого аналізу, проектування та реалізації складних програмних моделей мовою Python. Навчитися створювати класи для моделювання інженерних сутностей комп'ютерних систем, реалізовувати інкапсуляцію даних за допомогою механізму приховування атрибутів та дескрипторів властивостей (@property, @setter), застосовувати механізм наслідування з делегуванням конструктора через super(). Опанувати поліморфізм та перевантаження спеціальних методів моделі даних Python (dunder-методів: __init__, __str__, __repr__, __len__, __getitem__, __eq__, __lt__, __add__) для забезпечення природного математичного та системного синтаксису взаємодії між користувацькими об'єктами.

Студента III курсу КІ-24-1 групи
Ступінь вищої освіти бакалавр
(бакалавр, магістр)
Спеціальність 123 – «Комп’ютерна інженерія»
Освітня програма Комп’ютерна інженерія
Яцентюка Євгенія Олександровича

м. Кременчук 2026 рік

1. Мета роботи та технічна постановка задачі

Мета роботи: опанувати об’єктно-орієнтований аналіз, проєктування й реалізацію програмних моделей апаратних компонентів мовою Python: інкапсулювати стан об’єкта приватними атрибутами та властивостями @property із валідацією, будувати ієрархію класів через наслідування з делегуванням конструктора функцією super(), а також перевантажувати спеціальні методи моделі даних (__str__, __repr__, __eq__, __lt__, __add__, __len__, __getitem__), щоб об’єкти поводились природно у виразах з операторами, у функціях len() та sorted() і в індексації.

1.1. Індивідуальне завдання (варіант 2)

Базовий клас	**Похідний клас**	**Оператор + (__add__)**	**Оператор < (__lt__)**
Процесорне ядро CPUKernel (частота, IPC)	Векторний прискорювач SIMDKernel (розрядність)	сумування продуктивності ядер (MFLOPS)	порівняння за тактовою частотою

1.2. Технічні вимоги до класів

1) Базовий клас CPUKernel: приватні поля __kernel_id, __clock_frequency_ghz, __ipc та буфер телеметрії навантаження __load_history; валідація частоти та IPC через @property і @setter; методи __init__, __str__, __repr__, __eq__.
2) Похідний клас SIMDKernel: наслідування від CPUKernel, виклик super().__init__(), додаткове приватне поле __simd_width_bits із валідацією, нові властивості та перевизначення __str__, __repr__, __eq__.
3) Перевантаження операторів: __lt__ (порівняння за частотою), __add__ (сумування продуктивності ядер), а також протокол послідовностей __len__ і __getitem__ для доступу до буфера телеметрії.

1.3. Математична модель

Об’єкт моделюється абстрактним автоматом зі станом, де S — простір значень приватних атрибутів об’єкта, Σ — виклики методів і присвоєння властивостям, δ — функція переходів, s_0 — початковий стан, що встановлюється методом __init__:

$$M = \langle S, \Sigma, \delta, s_0 \rangle, \quad \delta : S \times \Sigma \to S$$

Для ядра CPUKernel стан S = (kernel_id, f, IPC, load_history), а інваріанти (0,1 ≤ f ≤ 8,0 ГГц; 0,1 ≤ IPC ≤ 1024; 0 ≤ навантаження ≤ 100 %) охороняються сетерами: перехід, що порушує інваріант, відхиляється винятком ValueError чи TypeError, і стан не змінюється. Пікова продуктивність ядра в одинарній точності:

$$P\ [MFLOPS] = f\ [ГГц] \cdot 1000 \cdot IPC \cdot L$$

де L = 1 для скалярного ядра CPUKernel та L = W / 32 для SIMD-ядра з регістром шириною W біт (кількість смуг FP32). Допустимі значення W: 128, 256 та 512 біт, отже L = 4, 8 або 16. Продуктивність є обчислюваною властивістю: у похідному класі її перевизначено, що й забезпечує поліморфізм.

Оператор додавання повертає зведене ядро, продуктивність якого дорівнює сумі продуктивностей доданків. Його параметри обираються так:

$$f_agg = \max(f_1, f_2), \quad IPC_agg = (P_1 + P_2) / (1000 \cdot f_agg), \quad \text{тоді} \quad P_agg = P_1 + P_2$$

Оператор порівняння визначається за частотою: a < b ⇔ f_a < f_b. Зверніть увагу: критерій варіанта не збігається з продуктивністю, тому ядро з меншою частотою може бути продуктивнішим (за рахунок SIMD).

2. UML-діаграма класів

Діаграму побудовано із зазначенням рівнів доступу (знак «−» — приватний, «+» — публічний), типів полів і сигнатур методів. Приватні поля з

префіксом __ доступні ззовні лише через властивості; позначки {readOnly}, {get, set} і {override} вказують на відсутність сетера, наявність гетера із сетером і перевизначення в похідному класі.

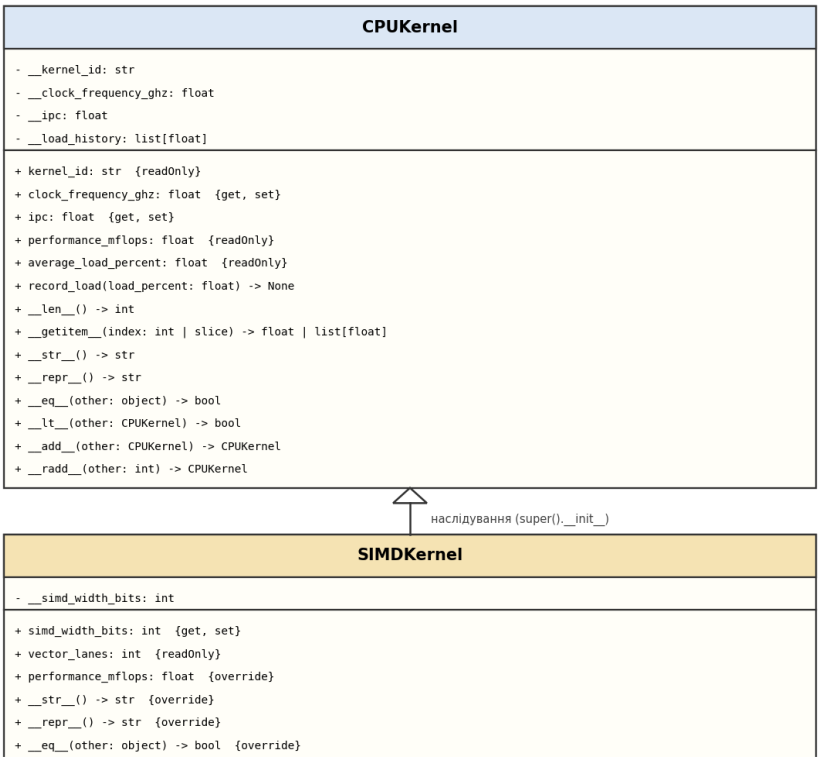


Рисунок 1 — UML-діаграма класів CPUKernel та SIMDKernel

2.1. Опис ролей членів класів

Елемент	Призначення
__kernel_id, kernel_id	ідентифікатор ядра; властивість лише для читання (сетера немає)
__clock_frequency_ghz, clock_frequency_ghz	тактова частота; сетер перевіряє тип (не bool) та діапазон 0,1…8,0 ГГц
__ipc, ipc	кількість інструкцій за такт; сетер перевіряє тип та діапазон 0,1…1024
__load_history, record_load()	буфер телеметрії навантаження 0…100 %; наповнюється лише через метод з валідацією
performance_mflops	обчислювана властивість P = f·1000·IPC; у SIMDKernel множиться на кількість смуг
__len__, __getitem__	протокол послідовностей: len(core), core[i], зрізи та ітерація по буферу телеметрії

Елемент	Призначення
__str__, __repr__	людиночитаний звіт та однозначний технічний рядок, за яким об’єкт можна відтворити
__eq__, __lt__	рівність за класом, id, частотою та IPC (для SIMD ще й розрядністю); порівняння за частотою
__add__, __radd__	сумування продуктивності; __radd__ дозволяє виклик sum() із початковим нулем
__simd_width_bits, vector_lanes	розрядність регістра (128/256/512) і кількість смуг FP32 L = W / 32

3. Особливості реалізації

3.1. Інкапсуляція та name mangling

Атрибути з префіксом __ компілятор перейменовує у форму _ІмяКласу__атрибут: у CPUKernel це _CPUKernel__ipc, у SIMDKernel — _SIMDKernel__simd_width_bits. Тому поле __ipc недоступне ззовні як obj.__ipc, а розрядність SIMD не конфліктує з полями базового класу. У конструкторі значення присвоюються через властивості (self.clock_frequency_ghz = …), тож інваріанти перевіряються вже під час створення об’єкта, а не лише при подальших змінах.

3.2. Диспетчеризація операторів

Вираз a + b виконується за протоколом: спершу викликається a.__add__(b); якщо результат дорівнює NotImplemented, інтерпретатор пробує b.__radd__(a); якщо й цей метод недоступний або повертає NotImplemented, генерується TypeError. Тому для непідтримуваного типу __add__ повертає NotImplemented (а не викликає виняток самостійно) і не повертає False. Метод __radd__ реалізовано лише для нуля: він потрібен вбудованій функції sum(), яка починає обчислення з виразу 0 + core.

Метод __add__ не змінює операндів, а створює нове ядро; його результат є CPUKernel і тому підтримує подальше додавання (ланцюжки a + b + c).

Перевірка isinstance(other, CPUKernel) дозволяє додавати ядра різних класів ієрархії, наприклад скалярне й SIMD.

3.3. Рівність, порівняння та хешованість

Метод __eq__ порівнює точний клас (type(self) is type(other)), ідентифікатор, частоту та IPC, причому дійсні числа порівнюються через math.isclose. Перевірка класу не дозволяє вважати рівними CPUKernel і SIMDKernel із однаковими базовими параметрами, а перевизначення в SIMDKernel додає порівняння розрядності. Для чужих типів повертається NotImplemented. Достатньо реалізувати лише __lt__: оператор > працює через відображений виклик __lt__ правого операнда, а sorted(), min() і max() потребують лише порівняння «менше».

Після визначення __eq__ без __hash__ Python встановлює __hash__ = None, тому об’єкти CPUKernel стають нехешованими й не можуть бути елементами множин або ключами словників. Для змінних об’єктів це правильна поведінка: хеш, що залежить від змінюваних полів, порушив би контракт хешованості.

4. Програмний код

4.1. Файл hardware_entities.py

```
"""
Лабораторна робота № 11. Модуль 1. Варіант 2.
Об'єктно-орієнтована модель процесорного ядра: базовий клас CPUKernel (тактова
частота, IPC) та похідний клас SIMDKernel (розрядність векторного регістра).
Інкапсуляція через @property/@setter, наслідування через super() та
перевантаження dunder-методів: __str__, __repr__, __eq__, __lt__, __add__,
__radd__, __len__, __getitem__.

Модель продуктивності (пікова, одинарна точність FP32):
    P [MFLOPS] = f [ГГц] * 1000 * IPC * L,   L = 1 для скалярного ядра,
                                             L = W / 32 для SIMD-ядра шириною W
біт
Семантика операторів варіанта 2:
    a + b  - сумування продуктивності ядер (MFLOPS); результат - нове зведене
ядро
    a < b  - порівняння за тактовою частотою
```

```
Освітній компонент: Програмування на Python.
"""

import math
from typing import List, Optional, Union

# Інженерні межі параметрів
CLOCK_MIN_GHZ, CLOCK_MAX_GHZ = 0.1, 8.0
# Верхня межа IPC дозволяє також ефективні значення зведених ядер (результат a + b)
IPC_MIN, IPC_MAX = 0.1, 1024.0
ALLOWED_SIMD_WIDTHS = (128, 256, 512)     # допустима розрядність векторного регістра, біт
FP32_LANE_BITS = 32                       # ширина однієї смуги FP32, біт
MFLOPS_PER_GHZ = 1000.0                   # 1 ГГц = 1000 МГц = 1000 млн операцій за секунду


def _require_number(value: object, name: str) -> float:
    """Перевіряє, що value є числом (bool відкидається, бо є підкласом int), і повертає float."""
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError(f"{name} повинна бути числовим значенням, отримано {type(value).__name__}.")
    return float(value)


class CPUKernel:
    """Базовий клас процесорного ядра: ідентифікатор, тактова частота, IPC, телеметрія навантаження."""

    def __init__(self, kernel_id: str, clock_frequency_ghz: float, ipc: float,
                 initial_load_history: Optional[List[float]] = None):
        name = str(kernel_id).strip()
        if not name:
            raise ValueError("Ідентифікатор ядра не може бути порожнім.")

        # Приватні атрибути: компілятор перейменовує їх у _CPUKernel__<ім'я> (name mangling)
        self.__kernel_id: str = name
        self.__clock_frequency_ghz: float = CLOCK_MIN_GHZ
        self.__ipc: float = IPC_MIN
        self.__load_history: List[float] = []

        # Ініціалізація через сетери запускає валідацію інваріантів уже в конструкторі
        self.clock_frequency_ghz = clock_frequency_ghz
        self.ipc = ipc
        for sample in (initial_load_history or []):
            self.record_load(sample)
```

```python
    # --- Властивості (інкапсуляція) ---

    @property
    def kernel_id(self) -> str:
        """Ідентифікатор ядра (лише читання: сетера немає)."""
        return self.__kernel_id

    @property
    def clock_frequency_ghz(self) -> float:
        """Гетер тактової частоти, ГГц."""
        return self.__clock_frequency_ghz

    @clock_frequency_ghz.setter
    def clock_frequency_ghz(self, value: float) -> None:
        """Сетер тактової частоти з перевіркою типу та інженерних меж."""
        freq = _require_number(value, "Тактова частота")
        if not (CLOCK_MIN_GHZ <= freq <= CLOCK_MAX_GHZ):
            raise ValueError(f"Тактова частота {freq} ГГц поза діапазоном "
                             f"[{CLOCK_MIN_GHZ}; {CLOCK_MAX_GHZ}] ГГц.")
        self.__clock_frequency_ghz = freq

    @property
    def ipc(self) -> float:
        """Гетер IPC (кількість інструкцій за такт)."""
        return self.__ipc

    @ipc.setter
    def ipc(self, value: float) -> None:
        """Сетер IPC з перевіркою типу та меж."""
        ipc_value = _require_number(value, "IPC")
        if not (IPC_MIN <= ipc_value <= IPC_MAX):
            raise ValueError(f"IPC {ipc_value} поза діапазоном [{IPC_MIN};
{IPC_MAX}].")
        self.__ipc = ipc_value

    @property
    def performance_mflops(self) -> float:
        """Обчислювана властивість: пікова продуктивність скалярного ядра,
MFLOPS."""
        return self.__clock_frequency_ghz * MFLOPS_PER_GHZ * self.__ipc

    @property
    def average_load_percent(self) -> float:
        """Середнє навантаження за накопиченою телеметрією, %."""
        if not self.__load_history:
            return 0.0
        return sum(self.__load_history) / len(self.__load_history)

    def record_load(self, load_percent: float) -> None:
```

```
        """Додає відлік навантаження (0..100 %) до внутрішнього буфера
телеметрії."""
        load = _require_number(load_percent, "Навантаження")
        if not (0.0 <= load <= 100.0):
            raise ValueError(f"Навантаження {load} % поза діапазоном [0; 100].")
        self.__load_history.append(load)

    # --- Протокол послідовностей: об'єкт поводиться як колекція відліків
телеметрії ---

    def __len__(self) -> int:
        """Кількість відліків у буфері: len(core)."""
        return len(self.__load_history)

    def __getitem__(self, index: Union[int, slice]) -> Union[float, List[float]]:
        """Індексація core[i] та зрізи core[a:b] делегуються внутрішньому
списку."""
        return self.__load_history[index]

    # --- Представлення ---

    def __repr__(self) -> str:
        """Однозначний технічний рядок: за ним об'єкт можна відтворити."""
        return (f"CPUKernel(kernel_id={self.__kernel_id!r}, "
                f"clock_frequency_ghz={self.__clock_frequency_ghz},
ipc={self.__ipc}, "
                f"initial_load_history={self.__load_history})")

    def __str__(self) -> str:
        """Форматований рядок для інженерних звітів."""
        return (f"CPU [{self.kernel_id}] | {self.clock_frequency_ghz:.2f} ГГц | "
                f"IPC {self.ipc:.3f} | {self.performance_mflops:.0f} MFLOPS | "
                f"Відліків: {len(self)}")

    # --- Порівняння ---

    def __eq__(self, other: object) -> bool:
        """
        Рівність: однакові клас, ідентифікатор, тактова частота та IPC.
        Перевірка точного класу запобігає рівності CPUKernel та SIMDKernel з
однаковими
        базовими параметрами. Для чужих типів повертаємо NotImplemented.
        """
        if not isinstance(other, CPUKernel):
            return NotImplemented
        return (type(self) is type(other)
                and self.kernel_id == other.kernel_id
                and math.isclose(self.clock_frequency_ghz,
other.clock_frequency_ghz, rel_tol=1e-9)
                and math.isclose(self.ipc, other.ipc, rel_tol=1e-9))
```

```python
    def __lt__(self, other: "CPUKernel") -> bool:
        """Критерій варіанта 2: «менше» означає меншу тактову частоту."""
        if not isinstance(other, CPUKernel):
            return NotImplemented
        return self.clock_frequency_ghz < other.clock_frequency_ghz

    # --- Арифметика: сумування продуктивності ---

    def __add__(self, other: "CPUKernel") -> "CPUKernel":
        """
        Сумування продуктивності ядер: повертає нове зведене ядро, продуктивність якого
        дорівнює P1 + P2. Тактова частота зведеного ядра - максимальна з двох, а ефективний
        IPC підбирається так, щоб P = f * 1000 * IPC збігалося із сумою. Буфери телеметрії
        конкатенуються. Операнди не змінюються.
        """
        if not isinstance(other, CPUKernel):
            return NotImplemented     # PVM спробує other.__radd__, а потім згенерує TypeError

        total_mflops = self.performance_mflops + other.performance_mflops
        merged_clock = max(self.clock_frequency_ghz, other.clock_frequency_ghz)
        effective_ipc = total_mflops / (merged_clock * MFLOPS_PER_GHZ)

        return CPUKernel(
            kernel_id=f"{self.kernel_id}+{other.kernel_id}",
            clock_frequency_ghz=merged_clock,
            ipc=effective_ipc,
            initial_load_history=self.__load_history + other.__load_history,
        )

    def __radd__(self, other: object) -> "CPUKernel":
        """
        Реверсивне додавання: потрібне вбудованій sum(), яка починає з 0 (0 + core).
        Для нуля повертаємо саме ядро, для інших типів - NotImplemented.
        """
        if isinstance(other, int) and not isinstance(other, bool) and other == 0:
            return self
        return NotImplemented


class SIMDKernel(CPUKernel):
    """Похідний клас: векторний прискорювач із розрядністю SIMD-регістра."""

    def __init__(self, kernel_id: str, clock_frequency_ghz: float, ipc: float,
                 simd_width_bits: int, initial_load_history: Optional[List[float]] = None):
        # Делегування ініціалізації спільної частини базовому класу
```

```
        super().__init__(kernel_id, clock_frequency_ghz, ipc,
initial_load_history)
        self.__simd_width_bits: int = ALLOWED_SIMD_WIDTHS[0]
        self.simd_width_bits = simd_width_bits

    @property
    def simd_width_bits(self) -> int:
        """Розрядність векторного регістра, біт."""
        return self.__simd_width_bits

    @simd_width_bits.setter
    def simd_width_bits(self, value: int) -> None:
        """Сетер розрядності: допустимі лише 128, 256 та 512 біт."""
        if isinstance(value, bool) or not isinstance(value, int):
            raise TypeError("Розрядність SIMD повинна бути цілим числом.")
        if value not in ALLOWED_SIMD_WIDTHS:
            raise ValueError(f"Розрядність {value} біт не підтримується:
допустимі {ALLOWED_SIMD_WIDTHS}.")
        self.__simd_width_bits = value

    @property
    def vector_lanes(self) -> int:
        """Кількість смуг FP32 у векторному регістрі: L = W / 32."""
        return self.__simd_width_bits // FP32_LANE_BITS

    @property
    def performance_mflops(self) -> float:
        """Поліморфне перевизначення: скалярна продуктивність, помножена на
кількість смуг."""
        return super().performance_mflops * self.vector_lanes

    def __repr__(self) -> str:
        """Технічний рядок похідного класу (перевизначає базовий)."""
        return (f"SIMDKernel(kernel_id={self.kernel_id!r}, "
                f"clock_frequency_ghz={self.clock_frequency_ghz}, ipc={self.ipc},
"
                f"simd_width_bits={self.__simd_width_bits}, "
                f"initial_load_history={self[:]})")

    def __str__(self) -> str:
        """Розширений звіт: базовий рядок плюс параметри векторного блоку."""
        return f"{super().__str__()} | SIMD: {self.__simd_width_bits} біт
({self.vector_lanes} смуг)"

    def __eq__(self, other: object) -> bool:
        """Рівність враховує також розрядність SIMD."""
        base_result = super().__eq__(other)
        if base_result is NotImplemented or not base_result:
            return base_result
        return self.simd_width_bits == other.simd_width_bits
```

4.2. Файл test_bench.py

```
"""
Лабораторна робота № 11. Модуль 2. Варіант 2.
Тестовий стенд верифікації інкапсуляції, поліморфізму та перевантаження
dunder-операторів ієрархії CPUKernel -> SIMDKernel.
Освітній компонент: Програмування на Python.
"""

import math
import sys
import time

from hardware_entities import CPUKernel, SIMDKernel

WIDTH = 90


def section(title: str) -> None:
    """Виводить роздільник і заголовок розділу тестового стенду."""
    print("-" * WIDTH)
    print(f"{title:^{WIDTH}}")
    print("-" * WIDTH)


def run_oop_testbench() -> list:
    """Виконує функціональне тестування моделі. Повертає журнал перевірок (опис,
результат)."""
    checks: list = []

    def check(description: str, condition: bool) -> None:
        """Реєструє перевірку інваріанта в журналі."""
        checks.append((description, bool(condition)))

    print("=" * WIDTH)
    print(f"{'ТЕСТОВИЙ СТЕНД ООП-МОДЕЛІ ПРОЦЕСОРНИХ ЯДЕР ТА DUNDER-
МЕТОДІВ':^{WIDTH}}")
    print("=" * WIDTH)

    # Початкові об'єкти базового та похідного класів
    core_a = CPUKernel("CORE-0", 3.2, 2.5, [35.0, 42.5, 38.0])
    core_b = CPUKernel("CORE-1", 2.8, 3.0, [60.0, 55.5])
    simd_a = SIMDKernel("SIMD-0", 3.0, 2.0, 256, [88.0, 91.5, 95.0, 90.5])
    simd_b = SIMDKernel("SIMD-1", 2.4, 1.5, 512, [70.0, 72.5])

    # 1. __str__ та __repr__
    section("1. СТРОКОВЕ ВІДОБРАЖЕННЯ (__str__ ТА __repr__)")
    for name, obj in (("core_a", core_a), ("core_b", core_b), ("simd_a", simd_a),
("simd_b", simd_b)):
        print(f"  print({name}): {obj}")
    print(f"  repr(core_a): {core_a!r}")
```

```
    print(f"  repr(simd_a): {simd_a!r}")
    rebuilt = eval(repr(simd_a))     # repr містить усе потрібне для відтворення об'єкта
    check("eval(repr(obj)) відтворює рівний об'єкт", rebuilt == simd_a)

    # 2. Інкапсуляція, name mangling, валідація сетерів
    section("2. ІНКАПСУЛЯЦІЯ: NAME MANGLING ТА ВАЛІДАЦІЯ (@property)")
    print(f"  Атрибути core_a:  {sorted(vars(core_a))}")
    print(f"  Атрибути simd_a:  {sorted(vars(simd_a))}")
    print(f"  core_a._CPUKernel__ipc = {core_a._CPUKernel__ipc}  (доступ через змінене ім'я)")
    try:
        _ = core_a.__ipc
    except AttributeError as err:
        print(f"  core_a.__ipc -> AttributeError: {err}")
        check("прямий доступ до __ipc заборонений", True)

    core_a.clock_frequency_ghz = 3.6
    print(f"  Частоту core_a змінено на {core_a.clock_frequency_ghz} ГГц через сетер")
    core_a.clock_frequency_ghz = 3.2          # повертаємо початкове значення
    for label, action in (
        ("частота 12.5 ГГц", lambda: setattr(core_a, "clock_frequency_ghz", 12.5)),
        ("частота 'fast'", lambda: setattr(core_a, "clock_frequency_ghz", "fast")),
        ("IPC = -1", lambda: setattr(core_a, "ipc", -1)),
        ("SIMD 192 біт", lambda: setattr(simd_a, "simd_width_bits", 192)),
        ("навантаження 120 %", lambda: core_a.record_load(120.0)),
    ):
        try:
            action()
            check(f"сетер відхилив {label}", False)
        except (ValueError, TypeError) as err:
            print(f"  [ЗАХИСТ СЕТЕРА] {label}: {type(err).__name__}: {err}")
            check(f"сетер відхилив {label}", True)
    try:
        core_a.kernel_id = "HACKED"
    except AttributeError as err:
        print(f"  kernel_id доступний лише для читання: {type(err).__name__}")
        check("kernel_id незмінний (немає сетера)", core_a.kernel_id == "CORE-0")

    # 3. Протокол послідовностей
    section("3. ПРОТОКОЛ ПОСЛІДОВНОСТЕЙ (__len__, __getitem__)")
    print(f"  len(simd_a) = {len(simd_a)}")
    print(f"  simd_a[0] = {simd_a[0]} %,  simd_a[-1] = {simd_a[-1]} %")
    print(f"  Зріз simd_a[1:3] = {simd_a[1:3]}")
    print(f"  Ітерація циклом for: {[load for load in simd_a]}")
    print(f"  Середнє навантаження simd_a: {simd_a.average_load_percent:.2f} %")
    simd_a.record_load(99.0)
```

```
    print(f"  Після record_load(99.0): len = {len(simd_a)}, simd_a[-1] =
{simd_a[-1]} %")
    try:
        _ = simd_a[10]
    except IndexError as err:
        print(f"  simd_a[10] -> IndexError: {err}")
        check("індекс поза межами дає IndexError", True)
    check("len() збігається з кількістю відліків", len(simd_a) == 5)

    # 4. __add__ та __radd__
    section("4. ПЕРЕВАНТАЖЕННЯ АРИФМЕТИКИ (__add__, __radd__)")
    print(f"  P(core_a) = {core_a.performance_mflops:.0f} MFLOPS,  P(core_b) =
{core_b.performance_mflops:.0f} MFLOPS")
    sum_cpu = core_a + core_b
    print(f"  core_a + core_b: {sum_cpu}")
    check("P(a + b) = P(a) + P(b) для скалярних ядер",
          math.isclose(sum_cpu.performance_mflops, core_a.performance_mflops +
core_b.performance_mflops))
    print(f"  P(simd_a) = {simd_a.performance_mflops:.0f} MFLOPS,  P(simd_b) =
{simd_b.performance_mflops:.0f} MFLOPS")
    sum_simd = simd_a + simd_b
    print(f"  simd_a + simd_b: {sum_simd}")
    check("P(simd_a + simd_b) = P(simd_a) + P(simd_b)",
          math.isclose(sum_simd.performance_mflops, simd_a.performance_mflops +
simd_b.performance_mflops))
    mixed = core_a + simd_a
    print(f"  core_a + simd_a: {mixed}")
    cluster_sum = sum([core_a, core_b, simd_a])      # sum() починає з 0: працює
__radd__
    print(f"  sum([core_a, core_b, simd_a]): {cluster_sum}")
    check("sum() дає сумарну продуктивність через __radd__",
          math.isclose(cluster_sum.performance_mflops,
                       core_a.performance_mflops + core_b.performance_mflops +
simd_a.performance_mflops))
    check("операнди після додавання не змінились", len(core_a) == 3 and
core_a.kernel_id == "CORE-0")
    try:
        _ = core_a + 5
    except TypeError as err:
        print(f"  core_a + 5 -> TypeError: {err}")
        check("непідтримуваний операнд дає TypeError", True)

    # 5. __eq__ та __lt__
    section("5. ОПЕРАТОРИ ПОРІВНЯННЯ (__eq__, __lt__)")
    twin = CPUKernel("CORE-0", 3.2, 2.5)
    fake_simd = SIMDKernel("CORE-0", 3.2, 2.5, 128)
    wide_twin = SIMDKernel("SIMD-0", 3.0, 2.0, 512)
    print(f"  core_a == twin (ті самі id, f, IPC):        {core_a == twin}")
    print(f"  core_a == core_b:                           {core_a == core_b}")
    print(f"  core_a != core_b:                           {core_a != core_b}")
    print(f"  core_a == SIMDKernel з тими ж id, f, IPC:   {core_a == fake_simd}")
```

```
    print(f"  simd_a == SIMD з іншою розрядністю:         {simd_a == wide_twin}")
    print(f"  core_a == 'CORE-0' (чужий тип):              {core_a == 'CORE-0'}")
    check("рівні за параметрами ядра рівні", core_a == twin)
    check("CPUKernel != SIMDKernel за типом", not (core_a == fake_simd))
    check("SIMD з різною розрядністю не рівні", not (simd_a == wide_twin))
    print(f"  Частоти, ГГц: core_a {core_a.clock_frequency_ghz}, core_b
{core_b.clock_frequency_ghz}, "
          f"simd_a {simd_a.clock_frequency_ghz}, simd_b
{simd_b.clock_frequency_ghz}")
    print(f"  core_b < core_a ?  {core_b < core_a}")
    print(f"  core_a < core_b ?  {core_a < core_b}")
    print(f"  simd_b < core_b ?  {simd_b < core_b}   (порівняння різних класів
ієрархії)")
    print(f"  core_a > core_b ?  {core_a > core_b}   (працює через відображення
__lt__)")
    check("__lt__ порівнює за частотою", core_b < core_a and not (core_a <
core_b))

    # 6. Поліморфізм та сортування
    section("6. ПОЛІМОРФІЗМ ТА СОРТУВАННЯ КЛАСТЕРА (sorted за тактовою
частотою)")
    cluster = [core_a, simd_a, core_b, simd_b]
    for rank, node in enumerate(sorted(cluster), start=1):
        print(f"  Ранг {rank}: {node}")
    print(f"  min(cluster) = {min(cluster).kernel_id},  max(cluster) =
{max(cluster).kernel_id}")
    best = max(cluster, key=lambda k: k.performance_mflops)
    print(f"  Найвища продуктивність (а не частота): {best.kernel_id} -
{best.performance_mflops:.0f} MFLOPS")
    check("порядок за частотою не збігається з порядком за продуктивністю",
          max(cluster) is not best)
    check("sorted() сортує за зростанням частоти",
          [k.clock_frequency_ghz for k in sorted(cluster)] == [2.4, 2.8, 3.0,
3.2])
    print("  Поліморфний виклик performance_mflops (різні реалізації в класах):")
    for node in cluster:
        print(f"    {type(node).__name__:<10} {node.kernel_id:<7}
{node.performance_mflops:>9.0f} MFLOPS")
    check("isinstance(simd_a, CPUKernel) істинне", isinstance(simd_a, CPUKernel))

    # 7. Хешованість
    section("7. ХЕШОВАНІСТЬ ПРИ ПЕРЕВИЗНАЧЕНОМУ __eq__")
    try:
        hash(core_a)
    except TypeError as err:
        print(f"  hash(core_a) -> TypeError: {err}")
        print("  Клас із власним __eq__ без __hash__ стає нехешованим: об'єкти
змінні.")
        check("об'єкт із __eq__ без __hash__ нехешований", True)

    # Підсумок перевірок
```

```
    passed = sum(1 for _, ok in checks if ok)
    print("=" * WIDTH)
    print(f"{'ПІДСУМОК ПЕРЕВІРОК ІНВАРІАНТІВ':^{WIDTH}}")
    print("-" * WIDTH)
    for description, ok in checks:
        print(f"  [{'OK' if ok else 'ПОМИЛКА'}] {description}")
    print("-" * WIDTH)
    print(f"  Пройдено перевірок: {passed} із {len(checks)}")
    print("=" * WIDTH + "\n")
    return checks


def main() -> None:
    """Головна точка входу лабораторної роботи."""
    t_start = time.perf_counter()
    checks = run_oop_testbench()
    t_elapsed = time.perf_counter() - t_start
    print(f"Час виконання об'єктно-орієнтованого тестування: {t_elapsed *
1000:.3f} мс\n")
    sys.exit(0 if all(ok for _, ok in checks) else 1)


if __name__ == "__main__":
    main()
```

5. Результати тестування

Запуск виконувався в середовищі Conda ce_lab_env командою python test_bench.py. Стенд має сім розділів і журнал із 23 автоматичних перевірок інваріантів; код завершення програми 0 свідчить, що всі перевірки пройдено.

```
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab11> (C:\Users\Janis\miniconda3\Scripts\activate) ; (conda activate
ce_lab_env)
PS C:\Users\Janis\Desktop\урок\Програмування на Python. Базовий рівень\lab11> & 'C:\Users\Janis\miniconda3\envs\ce_lab_env\python.exe' '"c:/U
sers/Janis/Desktop/урок/Програмування на Python. Базовий рівень/lab11/test_bench.py"'
================================================================================
            ТЕСТОВИЙ СТЕНД ООП-МОДЕЛІ ПРОЦЕСОРНИХ ЯДЕР ТА DUNDER-МЕТОДІВ
================================================================================
--------------------------------------------------------------------------------
                    1. СТРОКОВЕ ВІДОБРАЖЕННЯ (__str__ ТА __repr__)
--------------------------------------------------------------------------------
  print(core_a): CPU [CORE-0] | 3.20 ГГц | IPC 2.500 | 8000 MFLOPS | Відліків: 3
  print(core_b): CPU [CORE-1] | 2.80 ГГц | IPC 3.000 | 8400 MFLOPS | Відліків: 2
  print(simd_a): CPU [SIMD-0] | 3.00 ГГц | IPC 2.000 | 48000 MFLOPS | Відліків: 4 | SIMD: 256 біт (8 смуг)
  print(simd_b): CPU [SIMD-1] | 2.40 ГГц | IPC 1.500 | 57600 MFLOPS | Відліків: 2 | SIMD: 512 біт (16 смуг)
  repr(core_a): CPUKernel(kernel_id='CORE-0', clock_frequency_ghz=3.2, ipc=2.5, initial_load_history=[35.0, 42.5, 38.0])
  repr(simd_a): SIMDKernel(kernel_id='SIMD-0', clock_frequency_ghz=3.0, ipc=2.0, simd_width_bits=256, initial_load_history=[88.0, 91.5, 95.0,
 90.5])
--------------------------------------------------------------------------------
               2. ІНКАПСУЛЯЦІЯ: NAME MANGLING ТА ВАЛІДАЦІЯ (@property)
--------------------------------------------------------------------------------
  Атрибути core_a:  ['_CPUKernel__clock_frequency_ghz', '_CPUKernel__ipc', '_CPUKernel__kernel_id', '_CPUKernel__load_history']
  Атрибути simd_a:  ['_CPUKernel__clock_frequency_ghz', '_CPUKernel__ipc', '_CPUKernel__kernel_id', '_CPUKernel__load_history', '_SIMDKernel_
_simd_width_bits']
  core_a._CPUKernel__ipc = 2.5  (доступ через змінене ім'я)
  core_a.__ipc -> AttributeError: 'CPUKernel' object has no attribute '__ipc'
  Частоту core_a змінено на 3.6 ГГц через сетер
  [ЗАХИСТ СЕТЕРА] частота 12.5 ГГц: ValueError: Тактова частота 12.5 ГГц поза діапазоном [0.1; 8.0] ГГц.
  [ЗАХИСТ СЕТЕРА] частота 'fast': TypeError: Тактова частота повинна бути числовим значенням, отримано str.
  [ЗАХИСТ СЕТЕРА] IPC = -1: ValueError: IPC -1.0 поза діапазоном [0.1; 1024.0].
  [ЗАХИСТ СЕТЕРА] SIMD 192 біт: ValueError: Розрядність 192 біт не підтримується: допустимі (128, 256, 512).
  [ЗАХИСТ СЕТЕРА] навантаження 120 %: ValueError: Навантаження 120.0 % поза діапазоном [0; 100].
  kernel_id доступний лише для читання: AttributeError
--------------------------------------------------------------------------------
                  3. ПРОТОКОЛ ПОСЛІДОВНОСТЕЙ (__len__, __getitem__)
--------------------------------------------------------------------------------
  len(simd_a) = 4
  simd_a[0] = 88.0 %,  simd_a[-1] = 90.5 %
  Зріз simd_a[1:3] = [91.5, 95.0]
  Ітерація циклом for: [88.0, 91.5, 95.0, 90.5]
  Середнє навантаження simd_a: 91.25 %
  Після record_load(99.0): len = 5, simd_a[-1] = 99.0 %
  simd_a[10] -> IndexError: list index out of range
--------------------------------------------------------------------------------
                   4. ПЕРЕВАНТАЖЕННЯ АРИФМЕТИКИ (__add__, __radd__)
--------------------------------------------------------------------------------
  P(core_a) = 8000 MFLOPS,  P(core_b) = 8400 MFLOPS
  core_a + core_b: CPU [CORE-0+CORE-1] | 3.20 ГГц | IPC 5.125 | 16400 MFLOPS | Відліків: 5
  P(simd_a) = 48000 MFLOPS,  P(simd_b) = 57600 MFLOPS
  simd_a + simd_b: CPU [SIMD-0+SIMD-1] | 3.00 ГГц | IPC 35.200 | 105600 MFLOPS | Відліків: 7
  core_a + simd_a: CPU [CORE-0+SIMD-0] | 3.20 ГГц | IPC 17.500 | 56000 MFLOPS | Відліків: 8
  sum([core_a, core_b, simd_a]): CPU [CORE-0+CORE-1+SIMD-0] | 3.20 ГГц | IPC 20.125 | 64400 MFLOPS | Відліків: 10
  core_a + 5 -> TypeError: unsupported operand type(s) for +: 'CPUKernel' and 'int'
--------------------------------------------------------------------------------
                      5. ОПЕРАТОРИ ПОРІВНЯННЯ (__eq__, __lt__)
--------------------------------------------------------------------------------
  core_a == twin (ті самі id, f, IPC):       True
  core_a == core_b:                          False
  core_a != core_b:                          True
  core_a == SIMDKernel з тими ж id, f, IPC:  False
  simd_a == SIMD з іншою розрядністю:        False
  core_a == 'CORE-0' (чужий тип):            False
  Частоти, ГГц: core_a 3.2, core_b 2.8, simd_a 3.0, simd_b 2.4
  core_b < core_a ?  True
  core_a < core_b ?  False
  simd_b < core_b ?  True   (порівняння різних класів ієрархії)
  core_a > core_b ?  True   (працює через відображення __lt__)
--------------------------------------------------------------------------------
          6. ПОЛІМОРФІЗМ ТА СОРТУВАННЯ КЛАСТЕРА (sorted за тактовою частотою)
--------------------------------------------------------------------------------
  Ранг 1: CPU [SIMD-1] | 2.40 ГГц | IPC 1.500 | 57600 MFLOPS | Відліків: 2 | SIMD: 512 біт (16 смуг)
  Ранг 2: CPU [CORE-1] | 2.80 ГГц | IPC 3.000 | 8400 MFLOPS | Відліків: 2
  Ранг 3: CPU [SIMD-0] | 3.00 ГГц | IPC 2.000 | 48000 MFLOPS | Відліків: 5 | SIMD: 256 біт (8 смуг)
  Ранг 4: CPU [CORE-0] | 3.20 ГГц | IPC 2.500 | 8000 MFLOPS | Відліків: 3
  min(cluster) = SIMD-1,  max(cluster) = CORE-0
  Найвища продуктивність (а не частота): SIMD-1 - 57600 MFLOPS
  Поліморфний виклик performance_mflops (різні реалізації в класах):
    CPUKernel  CORE-0       8000 MFLOPS
    SIMDKernel SIMD-0      48000 MFLOPS
    CPUKernel  CORE-1       8400 MFLOPS
    SIMDKernel SIMD-1      57600 MFLOPS
--------------------------------------------------------------------------------
                     7. ХЕШОВАНІСТЬ ПРИ ПЕРЕВИЗНАЧЕНОМУ __eq__
--------------------------------------------------------------------------------
  hash(core_a) -> TypeError: unhashable type: 'CPUKernel'
  Клас із власним __eq__ без __hash__ стає нехешованим: об'єкти змінні.
================================================================================
                            ПІДСУМОК ПЕРЕВІРОК ІНВАРІАНТІВ
--------------------------------------------------------------------------------
  [OK] eval(repr(obj)) відтворює рівний об'єкт
  [OK] прямий доступ до __ipc заборонений
  [OK] сетер відхилив частота 12.5 ГГц
  [OK] сетер відхилив частота 'fast'
  [OK] сетер відхилив IPC = -1
  [OK] сетер відхилив SIMD 192 біт
  [OK] сетер відхилив навантаження 120 %
  [OK] kernel_id незмінний (немає сетера)
  [OK] індекс поза межами дає IndexError
  [OK] len() збігається з кількістю відліків
  [OK] P(a + b) = P(a) + P(b) для скалярних ядер
  [OK] P(simd_a + simd_b) = P(simd_a) + P(simd_b)
  [OK] sum() дає сумарну продуктивність через __radd__
  [OK] операнди після додавання не змінились
  [OK] непідтримуваний операнд дає TypeError
  [OK] рівні за параметрами ядра рівні
  [OK] CPUKernel != SIMDKernel за типом
```

```
  [OK] SIMD з різною розрядністю не рівні
  [OK] __lt__ порівнює за частотою
  [OK] порядок за частотою не збігається з порядком за продуктивністю
  [OK] sorted() сортує за зростанням частоти
  [OK] isinstance(simd_a, CPUKernel) істинне
  [OK] об'єкт із __eq__ без __hash__ нехешований
--------------------------------------------------------------------------------
  Пройдено перевірок: 23 із 23
================================================================================

Час виконання об'єктно-орієнтованого тестування: 6.089 мс
```

Рисунок 2 — Результат роботи тестового стенда ООП-моделі процесорних ядер

5.1. Аналіз результатів

Продуктивність. Для core_a P = 3,2 · 1000 · 2,5 = 8000 MFLOPS; для core_b P = 2,8 · 1000 · 3,0 = 8400 MFLOPS. Для SIMD-ядер множник L = W / 32: simd_a (256 біт, 8 смуг) дає 3,0 · 1000 · 2,0 · 8 = 48000 MFLOPS, simd_b (512 біт, 16 смуг) — 2,4 · 1000 · 1,5 · 16 = 57600 MFLOPS. Рядок виведення SIMD-ядер автоматично містить розрядність і кількість смуг, бо SIMDKernel розширює базовий __str__ через super().__str__().

Інкапсуляція. У словнику атрибутів core_a присутні лише змінені імена _CPUKernel__…, а в simd_a додатково _SIMDKernel__simd_width_bits. Звернення core_a.__ipc дає AttributeError, а core_a._CPUKernel__ipc повертає 2,5. Усі п’ять некоректних присвоєнь відхилено сетерами: частота 12,5 ГГц та IPC = −1 дали ValueError, частота «fast» — TypeError, SIMD 192 біт та навантаження 120 % — ValueError. Спроба змінити kernel_id завершилась AttributeError, бо сетера немає.

Протокол послідовностей. Для simd_a len() повертає 4, simd_a[0] = 88,0 %, від’ємний індекс simd_a[−1] = 90,5 %, зріз simd_a[1:3] = [91,5; 95,0]; цикл for працює навіть без __iter__, оскільки інтерпретатор перебирає індекси 0, 1, 2 … через __getitem__ до появи IndexError. Після record_load(99,0) довжина зросла до 5, а індекс 10 дав IndexError.

Арифметика. core_a + core_b дає зведене ядро з P = 8000 + 8400 = 16400 MFLOPS, f = 3,2 ГГц та IPC = 16400 / 3200 = 5,125. Для simd_a + simd_b P = 48000 + 57600 = 105600 MFLOPS (IPC = 35,2); для різнотипних core_a + simd_a

P = 56000 MFLOPS. Вираз sum([core_a, core_b, simd_a]) дає 64400 MFLOPS (8000 + 8400 + 48000) завдяки __radd__ для початкового нуля. Кількість відліків у сумі дорівнює сумі довжин буферів (3 + 2 + 5 = 10). Вираз core_a + 5 завершується TypeError, оскільки обидва методи (__add__ та __radd__ числа) повернули NotImplemented.

Порівняння. Ядра з однаковими id, частотою та IPC рівні (core_a == twin), а ядра з різними параметрами — ні; оператор != працює автоматично як заперечення __eq__. CPUKernel і SIMDKernel із однаковими базовими параметрами не рівні через перевірку класу, а два SIMD-ядра з різною розрядністю також не рівні. Порівняння із чужим типом (рядком) дає False. Оператор < порівнює частоти (core_b 2,8 < core_a 3,2), а оператор > для core_a > core_b працює через відображений __lt__.

Поліморфізм. Сортування кластера за зростанням частоти дає порядок SIMD-1 (2,4 ГГц), CORE-1 (2,8), SIMD-0 (3,0), CORE-0 (3,2). Водночас найвищу продуктивність (57600 MFLOPS) має саме SIMD-1, тобто ядро з найнижчою частотою; порядок за частотою не збігається з порядком за продуктивністю, що й підтверджено перевіркою. Цикл по змішаному списку викликає performance_mflops без перевірки типу, і кожен об’єкт використовує власну реалізацію властивості. Спроба hash(core_a) викликає TypeError, що підтверджує нехешованість класу з власним __eq__.

Повідомлення Python про спробу змінити kernel_id у звіті не наводиться: виводиться лише назва типу винятку AttributeError, оскільки текст повідомлення відрізняється між версіями Python 3.10 та 3.11 і новішими, а решта виведення від версії не залежить.

6. Посилання на віддалений репозиторій

Профіль: https://github.com/kefir4ikk
Репозиторій лабораторної роботи: https://github.com/kefir4ikk/progPython_YevgeniiYatsentiuk/tree/master/labs/lab11

7. Висновки

У ході лабораторної роботи спроєктовано й реалізовано ієрархію CPUKernel → SIMDKernel для варіанта 2. Парадигма ООП дозволила описати процесорне ядро як об’єкт із прихованим станом і керованим інтерфейсом: приватні атрибути __ та властивості @property із сетерами гарантують, що частота, IPC, розрядність SIMD і навантаження завжди перебувають у фізично допустимих межах, а некоректне значення виявляється в момент присвоєння, а не пізніше в обчисленнях. Ініціалізація через сетери в конструкторі робить інваріанти чинними від самого створення об’єкта.

Наслідування з делегуванням super().__init__() дозволило повторно використати базову логіку, а перевизначення обчислюваної властивості performance_mflops, а також методів __str__, __repr__ і __eq__ забезпечило поліморфізм: код, що працює з CPUKernel, без змін обробляє SIMDKernel. Dunder-методи зробили запис виразним і природним: ядра сумуються оператором +, порівнюються операторами == та <, сортуються функцією sorted(), а буфер телеметрії доступний як колекція через len(), індексацію, зрізи та ітерацію. Реалізація __radd__ підключила вбудовану функцію sum(), а повернення NotImplemented замість винятку дозволило інтерпретатору коректно диспетчеризувати операцію і зрештою згенерувати TypeError для непідтримуваних типів.

Тестування виявило важливі тонкощі, які враховано в моделі: критерій порівняння «за частотою» не збігається з продуктивністю, тому сортування та пошук мінімуму/максимуму потрібно інтерпретувати саме за частотою; клас із власним __eq__ стає нехешованим; результат додавання має бути новим об’єктом, щоб не змінювати операнди. Модуль test_bench.py із журналом із 23 автоматичних перевірок демонструє, що всі інваріанти виконуються.

Контрольні питання

1. Як влаштований механізм спотворення імен (name mangling) у CPython? Чому приватний атрибут __sampling_rate_hz недоступний як obj.__sampling_rate_hz, але доступний через obj._SignalProcessorUnit__sampling_rate_hz?

Під час компіляції тіла класу кожен ідентифікатор виду __name (щонайменше два початкові підкреслення й не більше одного кінцевого) замінюється на _ІмяКласу__name. Заміна виконується для всіх звернень у коді цього класу, зокрема self.__name, тому всередині класу все працює прозоро, а значення зберігається в obj.__dict__ під зміненим ключем. Код поза класом не підлягає перейменуванню: вираз obj.__name шукає атрибут із буквальним іменем __name, якого в об’єкті немає, і дає AttributeError. Змінене ім’я _ІмяКласу__name залишається звичайним атрибутом і доступне, тому це не захист, а засіб уникнути випадкового перезапису полів у підкласах: у підкласі self.__x перетворюється на _Підклас__x, а не на поле базового класу. Імена, що закінчуються двома підкресленнями (__init__), не змінюються.

2. Поясніть принципову різницю між спеціальними методами __str__() та __repr__(). У яких системних ситуаціях інтерпретатор автоматично викликає __repr__ замість __str__?

Метод __str__ повертає зручне для людини представлення (для звітів та виведення користувачу), а __repr__ — однозначний технічний рядок для розробника, за яким бажано можна відтворити об’єкт: eval(repr(x)) == x. Функції print(), str() та форматування використовують __str__; якщо він не визначений, використовується __repr__ (реалізація object.__str__ викликає repr). Виклик __repr__ відбувається при виведенні значення в інтерактивній консолі, у функції repr(), при форматуванні специфікатором !r або %r, у повідомленнях налагоджувача, трасувань та тверджень тестів, а також для елементів контейнерів: print([core]) виводить елементи списку саме через __repr__.

3. Опишіть роботу протоколу дескрипторів, що лежить в основі декораторів @property та @setter. Яким чином сетер властивості запобігає переходу об'єкта в некоректний стан?

Дескриптор — це об’єкт, що визначає в класі методи __get__, __set__ та/або __delete__. Клас property є дескриптором даних: пошук атрибута спершу шукає в типі об’єкта (за MRO) дескриптор даних, і він має пріоритет над записом в obj.__dict__. Читання obj.x викликає property.__get__, яке виконує функцію-гетер, а присвоєння obj.x = v — property.__set__, яке виконує функцію-сетер; без сетера присвоєння дає AttributeError. Сетер перед записом у приватне поле перевіряє тип і діапазон і генерує виняток для некоректного значення, тому поле лишається в старому допустимому стані. Оскільки конструктор також присвоює значення через властивості, інваріанти діють від першого моменту життя об’єкта.

4. У чому полягає роль функції super() при наслідуванні класів? Чому при множинному наслідуванні виклик super() є обов'язковим для дотримання лінеаризації MRO за алгоритмом C3?

Функція super() повертає проксі-об’єкт, який делегує пошук атрибутів наступному класу в порядку розв’язання методів (MRO) після поточного, а не обов’язково безпосередньому батьку. У цій роботі super().__init__() ініціалізує спільну частину стану, а super().performance_mflops і super().__str__() розширюють поведінку базового класу. Алгоритм лінеаризації C3 будує для класу єдиний монотонний порядок усіх предків (Class.__mro__). Якщо кожен клас у ланцюжку викликає super().__init__(), то ініціалізатори виконуються рівно по одному разу в порядку MRO (кооперативне наслідування). Прямий виклик Base.__init__(self) у ромбоподібній ієрархії може пропустити сусідній клас або ініціалізувати спільного предка двічі.

5. Що повинна повертати функція перевантаження арифметичного оператора __add__(self, other), якщо переданий операнд other має непідтримуваний тип, і як віртуальна машина PVM обробляє значення NotImplemented?

Метод має повернути спеціальне значення NotImplemented (а не виняток і не False). Значення NotImplemented сигналізує інтерпретатору, що цей операнд не вміє виконати операцію. Тоді інтерпретатор намагається викликати відображений метод правого операнда other.__radd__(self); якщо right-операнд є екземпляром підкласу лівого типу й перевизначає відображений метод, його викликають першим. Якщо й відображений метод повертає NotImplemented або відсутній, інтерпретатор сам генерує TypeError «unsupported operand type(s) for +». Саме так вираз core_a + 5 у цій роботі завершується TypeError, а sum() працює завдяки реалізованому __radd__ для початкового нуля.

6. Розкрийте концепцію качиної типізації (Duck Typing) у контексті поліморфізму Python. Чому функція sorted() здатна впорядковувати список користувацьких об'єктів без явної реалізації спеціальних інтерфейсів?

Качина типізація — підхід, за яким придатність об’єкта визначається наявністю потрібної поведінки (методів), а не його оголошеним типом чи спільним предком: «якщо воно ходить як качка й крякає як качка, то це качка». Функція sorted() не вимагає успадкування від спеціального інтерфейсу: вона лише порівнює елементи виразом a < b, що викликає метод __lt__ першого операнда. Тому будь-які об’єкти, які реалізують __lt__, сортуються, причому це можуть бути об’єкти різних класів, як CPUKernel та SIMDKernel у цій роботі (SIMDKernel успадковує __lt__). Якщо методу немає, виникає TypeError «’<’ not supported between instances». Аналогічно len() вимагає лише __len__, а цикл for може працювати через __getitem__. Для повного набору порівнянь за одним методом можна використати functools.total_ordering.